



北京航空航天大学

BEIHANG UNIVERSITY

Pthreads 编程实验报告

《并行程序设计 A》课程报告

学 院： 软件学院

班 级： ZY25211

姓 名： 刘益洲

学 号： ZY2521110

目录

一、 实验环境	1
二、 实验内容与方法	2
(一) 实验内容	2
(二) 实验方法	2
三、 实验过程	2
(一) 并行快速排序实现过程	2
(二) 计时与重复实验流程	3
(三) 程序编译与运行	4
四、 实验结果与分析	4

一、实验环境

本实验在表 1 所列硬件与系统环境下完成，以下信息由本机在实验数据采集时通过 `system_profiler` 与 `sysctl` 读取。

表 1 实验用机硬件与操作系统环境

项目	规格或说明
机型名称	MacBook Pro
机型标识	Mac16,8
处理器	Apple M4 Pro；共 12 核（8 个性能核心 + 4 个能效核心）
逻辑处理器数量	12
内存容量	24 GB
图形处理器	Apple M4 Pro 集成 GPU（16 核，支持 Metal 3）
操作系统	macOS 15.4.1（构建号 24E263）

编译、链接与 `pthread`（POSIX Threads）运行时所依赖的主要软件版本见表 2（由本机在实验环境整理阶段通过终端输出查询得到）。

表 2 编译器与 `pthread` 相关软件环境

项目	规格或说明
C/C++ 编译器	Apple clang 17.0.0（clang-1700.0.13.5）
编译目标三元组	arm64-apple-darwin24.4.0
Xcode 版本	16.4（16F6）
Command Line Tools	CLTools_Executables 16.4.0.0.1.1747106510
macOS SDK	15.5
构建工具	GNU Make 3.81
Darwin 内核版本	24.4.0（xnu-11417.101.15~117）
pthread 运行时	来自 macOS 系统库 <code>libsystem_pthread</code> （current version 536.0.0；随 macOS 版本演进）
编译优化等级	-O3

二、实验内容与方法

（一）实验内容

本次作业要求使用 `pthread`s 实现并行快速排序程序，编程语言为 C/C++。待排序数组长度不小于 2000 万，数组元素类型为双精度浮点数；至少测试线程数为 1、2、3、4（如平台允许可更多）。测试时需保持数据集一致，并在统计排序耗时排除准备数据（如 I/O 等）的时间。

（二）实验方法

`pthread`s (POSIX Threads) 是 POSIX 标准定义的线程编程接口，提供线程创建与回收（如 `pthread_create`/`pthread_join`）、互斥锁与条件变量等原语，用于在同一进程内实现共享内存并行。

快速排序的核心是对区间进行划分（`partition`）并递归处理左右子区间。并行化时将待排序子区间视为任务，线程从任务队列或任务栈中取出区间执行划分，并将新的子区间继续作为任务提交。当子区间规模较小时转为顺序排序，以控制并行粒度与调度开销。

实现方面，我们编写并行排序程序文件 `pqs.c`，支持通过命令行参数指定数组长度与线程数，并使用 `Makefile` 以 `-O3` 优化编译生成可执行文件。实验阶段通过脚本 `bench_pqs.sh` 自动化运行，脚本顶部以变量配置实验参数，亦可通过同名环境变量覆盖默认值，并在缺失可执行文件时尝试执行 `make` 生成 `pqs`。脚本依次调用 `pqs`，考察线程数为 1、2、3、4、6、8 且数组长度为 200000000 时的运行时间，每次运行固定传入 `--seed` 与 `--threshold` 以保证不同线程数对比时数据生成与粒度策略一致。每组配置重复运行 5 次，从输出中读取 `time_sec` 字段取平均值，并将明细写入 `results/pqs_bench.csv`。

三、实验过程

（一）并行快速排序实现过程

程序采用固定数量的工作线程，并维护一个存放待排序区间的任务栈。初始时将整段区间作为任务入栈，工作线程循环取出任务处理。线程的取任务与等待逻辑如代码 1 所示。

```
1 while (!ctx->stop && ctx->stack_sz == 0 && ctx->tasks_total > 0) {  
2   pthread_cond_wait(&ctx->cv, &ctx->mu);
```

```

3 }
4 bool ok = pop_task_locked(ctx, &t);
5 pthread_mutex_unlock(&ctx->mu);
6 if (!ok) continue;
7 process_task(ctx, t);

```

代码 1: 工作线程主循环 (节选)

每个任务对应数组的一个子区间。处理任务时先执行一次划分, 将左右子区间进一步作为任务提交, 并对较小区间进行继续处理。为控制粒度, 当区间长度不大于阈值时转为顺序排序。对应逻辑如代码 2 所示。

```

1 while ((t.hi - t.lo) > ctx->seq_threshold) {
2     size_t m = partition_range(ctx->arr, t.lo, t.hi);
3     Task left = (Task){.lo = t.lo, .hi = m};
4     Task right = (Task){.lo = m, .hi = t.hi};
5     pthread_mutex_lock(&ctx->mu);
6     if (!ctx->stop) push_task_locked(ctx, large);
7     pthread_mutex_unlock(&ctx->mu);
8     t = small;
9 }
10 sort_sequential(ctx->arr, t.lo, t.hi);

```

代码 2: 区间任务处理与粒度控制 (节选)

(二) 计时与重复实验流程

数组数据在计时开始前生成, 以保证统计的主要为并行排序过程。计时采用 `CLOCK_MONOTONIC` 并围绕线程创建与回收区间进行统计, 相关代码如代码 3 所示。

```

1 double t0 = now_sec();
2 for (size_t i = 0; i < threads; i++) {
3     pthread_create(&tids[i], NULL, worker_main, &ctx);
4 }
5 for (size_t i = 0; i < threads; i++) {
6     pthread_join(tids[i], NULL);
7 }
8 double t1 = now_sec();

```

代码 3: 计时代码 (节选)

实验数据采集由脚本 `bench_pqs.sh` 自动化完成, 固定数组长度为 200000000, 线程数分别取 1、2、3、4、6、8。每组配置重复执行 5 次, 从 pqs 输出中解析 `time_sec` 作为单次耗时, 并计算算术平均用于后续结果分析, 同时将每次运行的记录写入 CSV

文件 `results/pqs_bench.csv`。

（三）程序编译与运行

本实验程序位于 `pqs.c`，使用 `Makefile` 进行构建。编译规则较为简单，如代码 4 所示。

```
1 CC ?= clang
2 CFLAGS ?= -O3 -std=c11 -Wall -Wextra -Wpedantic
3 LDLIBS ?= -pthread
4
5 $(TARGET): $(SRC)
6     $(CC) $(CFLAGS) $(LDFLAGS) -o $@ $^ $(LDLIBS)
```

代码 4: 程序构建规则

程序通过命令行参数指定数组长度与线程数，并可选指定随机种子与阈值。参数解析逻辑如代码 5 所示。

```
1 if (!strcmp(arg, "-n") || !strcmp(arg, "--n")) {
2     n = (size_t)v;
3 } else if (!strcmp(arg, "-t") || !strcmp(arg, "--threads")) {
4     threads = (size_t)v;
5 } else if (!strcmp(arg, "--seed")) {
6     seed = v ? v : 1;
7 } else if (!strcmp(arg, "--threshold")) {
8     threshold = (size_t)v;
9 }
```

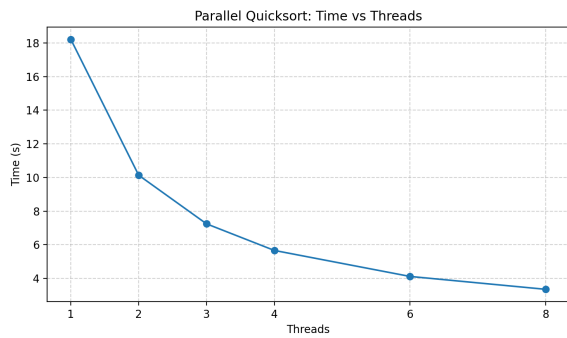
代码 5: 命令行参数解析（节选）

四、实验结果与分析

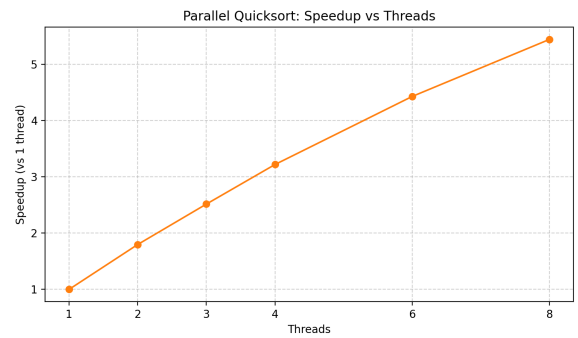
本节给出 `pthread` 并行快速排序在本机环境下的实测结果，并围绕表 3 对趋势进行解释。实验固定数组长度为 200000000，元素类型为双精度浮点数，随机种子取 1，顺序排序阈值取 16384，从而保证不同线程数对比时数据生成方式与并行粒度策略一致。线程数分别取 1、2、3、4、6、8，每个配置重复运行 5 次，取算术平均作为该配置下的排序耗时 T_p 。在此基础上定义相对单线程的加速比 $S_p = T_1/T_p$ ，以及并行效率 $E_p = S_p/p$ ，用于衡量“每增加一个线程带来的边际收益”。

表 3 并行快速排序性能结果（平均值，单位秒）

线程数 p	平均耗时 T_p	加速比 S_p	并行效率 E_p
1	18.212417	1.000	1.000
2	10.134970	1.797	0.898
3	7.242837	2.515	0.838
4	5.660566	3.217	0.804
6	4.113904	4.427	0.738
8	3.347781	5.440	0.680



(a) Time vs threads



(b) Speedup vs threads

图 1 Measured performance curves (English plot labels)

从表 3 的总体趋势看， T_p 随线程数增加单调下降，说明在该数据规模下并行化能够持续缩短排序时间。与此同时， S_p 随线程数增加而上升，但始终低于理想线性加速，且 E_p 从 2 线程的约 0.90 逐步下降到 8 线程的约 0.68，表明随着并行度提高，任务切分、同步与调度等开销在总时间中的占比上升，同时多线程对内存子系统的竞争也会削弱“每线程等比例贡献计算量”的理想假设。

图 1a 将 T_p 随线程数变化可视化。曲线整体呈下凹形态，意味着增加线程带来的边际收益逐步变小。例如从 4 线程到 6 线程，平均耗时下降约 1.55 秒，而从 6 线程到 8 线程仅下降约 0.77 秒，说明在更高并行度阶段更容易出现收益递减。图 1b 给出 S_p 的变化，曲线持续上升但斜率逐渐放缓，与“收益递减”的观察一致。若将相邻线程数之间的加速比增量进行对比，也可以看到从 2 到 3、从 3 到 4 的提升相对更明显，而从 6 到 8 的提升相对温和，这与任务栈竞争、缓存局部性变化以及系统调度等因素共同作用下的经验现象相符。

需要强调的是，本实验计时区间包含线程创建与回收开销，因此当线程数较少时，这部分固定成本在总时间中占比更高，而当线程数较多时，排序主体计算占比上升，曲线形态也会相应变化。若要进一步量化随机性，可在相同固定参数下增加重复次数，并

同时汇报方差或置信区间，以便区分“算法与实现层面的可解释因素”和“测量噪声”。